

1. Design Read and Replace a text in a word document using UiPath

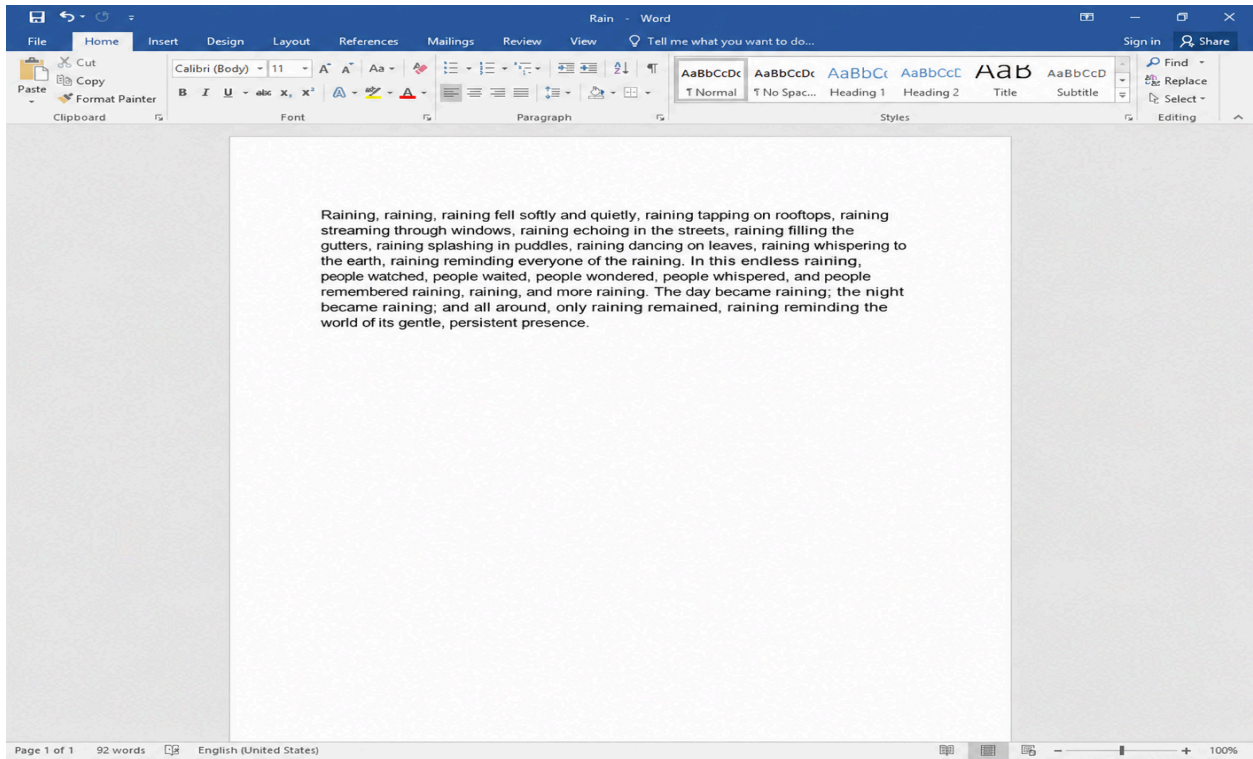
AIM :

To design, read and replace a text in a word document using UiPath.

ALGORITHM:

- Step 1:** Start the UiPath automation process and initialize the required variables.
- Step 2:** Use Word Application Scope to open the input Word document.
- Step 3:** Inside Word Application Scope, add the Replace Text activity.
- Step 4:** In Replace Text, enter "rain" in the Search field.
- Step 5:** Enter "raining" in the Replace field and set Replace All to Yes.
- Step 6:** Add the Append Text activity to add the required text to the document.
- Step 7:** In Append Text, enter "This text was added by UiPath" and set New Line to Yes.
- Step 8:** Add the Add Picture activity, select the image file, and set Position to End.
- Step 9:** Run the workflow and verify the modified Word document .

WORKFLOW:



Main

Main Sequence

Expand AllCollapse All

Word Application Scope

File path *

"C:\Users\USER\Desktop\ReadReplace.docx"

...

Do

Replace Text

Search *

"rain"

...

Replace *

"raining"

...

Replace all *

Yes

Append Text

Text *

"This text was added by UiPath v"

...

New line *

Yes

Add Picture

Picture to insert *

"C:\Users\USER\Desktop\anime.jpg"

...

Insert relative to *

Document

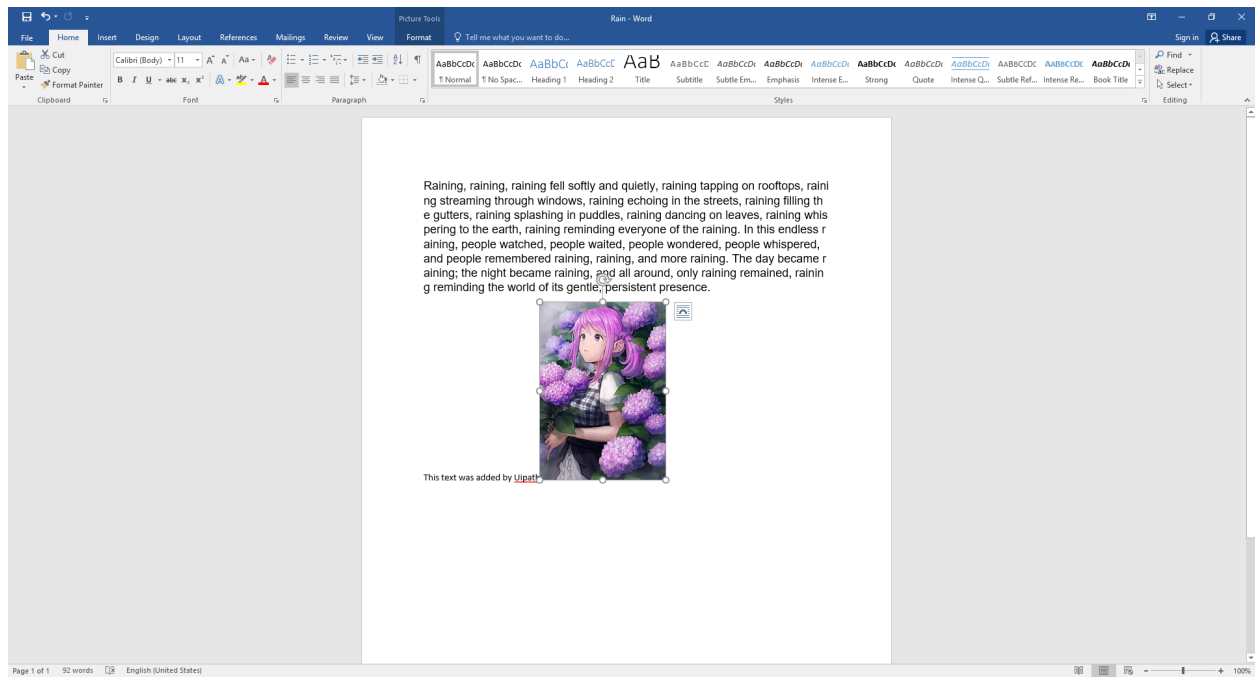
...

Position

End

...

OUTPUT:



RESULT:

Thus, the program has been executed successfully and output is verified.

2. Design a bot that automates the on boarding process for new employees.

AIM:

To design a bot that automates the on boarding process for new employees.

ALGORITHMS:

Step 1: Start the UiPath automation process and initialize the required variables.

Step 2: Start the UiPath automation process and initialize the required variables.

Step 3: Use another Assign activity to store the employee email information.

Step 4: Use an Assign activity to store the employee department details.

Step 5: Use the Assign activity to check whether the employee belongs to the IT department.

Step 6: Add an If activity to verify the employee department.

Step 7: In the Then section, add a Message Box activity to display the employee welcome message.

Step 8: In the Else section, add another Message Box activity to display the manual onboarding requirement message.

Step 9: Run the workflow and verify the employee onboarding process output.

WORKFLOW:

 Data Manager

 Output

 Errors

 Markers

 Terminal

 Variables

 Arguments

 Namespaces

 Connections

Name	 Data Type	Scope	 Default Value
Create variable			
{x} empName	String	Main	{ }
{x} email	String	Main	{ }
{x} department	String	Main	{ }
{x} isITDept	Boolean	Main	{ } False

+

Assign

To variable *

Out

{}

 empName

+

Set value *

{}

 "John"

↵

+

↓

Assign

To variable *

Out

{}

 email

+

Set value *

{}

 "john123@gmail.com"

↵

+

↓

Assign

To variable *

Out

{}

 department

+

Set value *

{}

 "IT Support"

↵

+

↓

Assign

To variable *

Out

{}

 isITDept

+

Set value *

{}

 False

↵

+

↓

 **Assign**  

To variable * Out

`{}` isITDept 

Set value *

`{}` department.Contains("IT") 

↓

 **If**  

Condition *

`{}` department = "IT Support" 

↳ **Then**  



 **Message Box**  

Text *

`{}` "Welcome " + empName 



↳ **Else**  



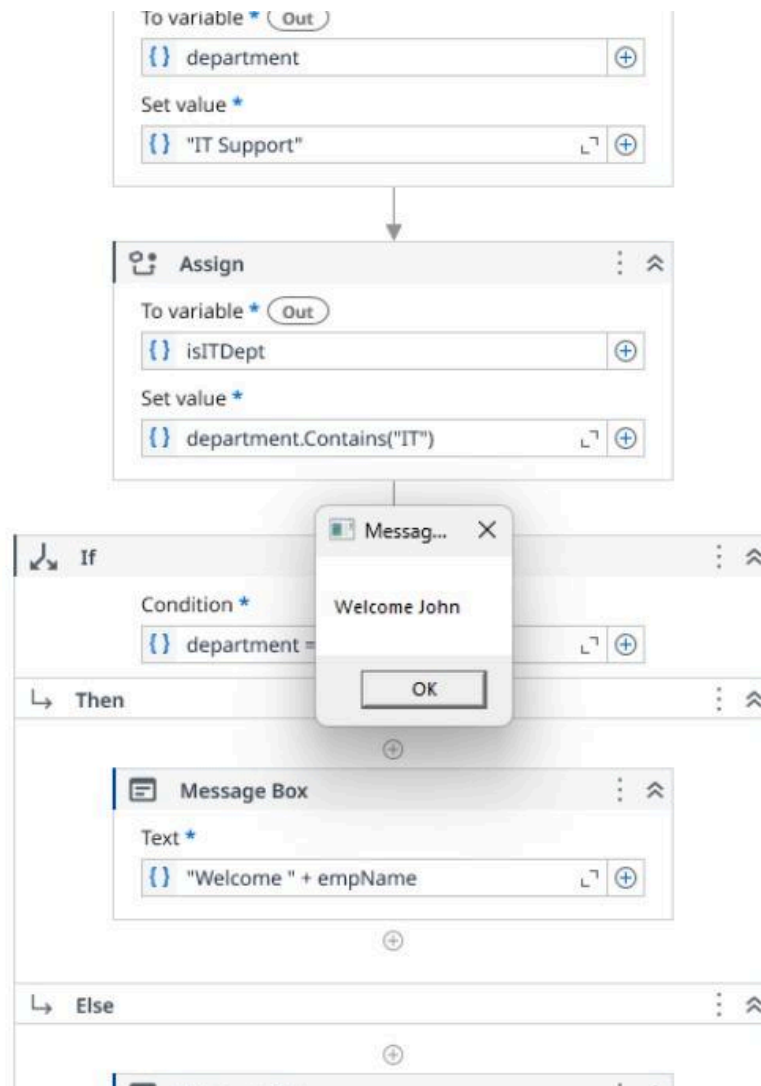
 **Message Box**  

Text *

`{}` "Manual Onboarding Required" 



OUTPUT:



RESULT:

Thus, the program has been executed successfully and output is verified.

3. Create s Screen scraping using UiPath.

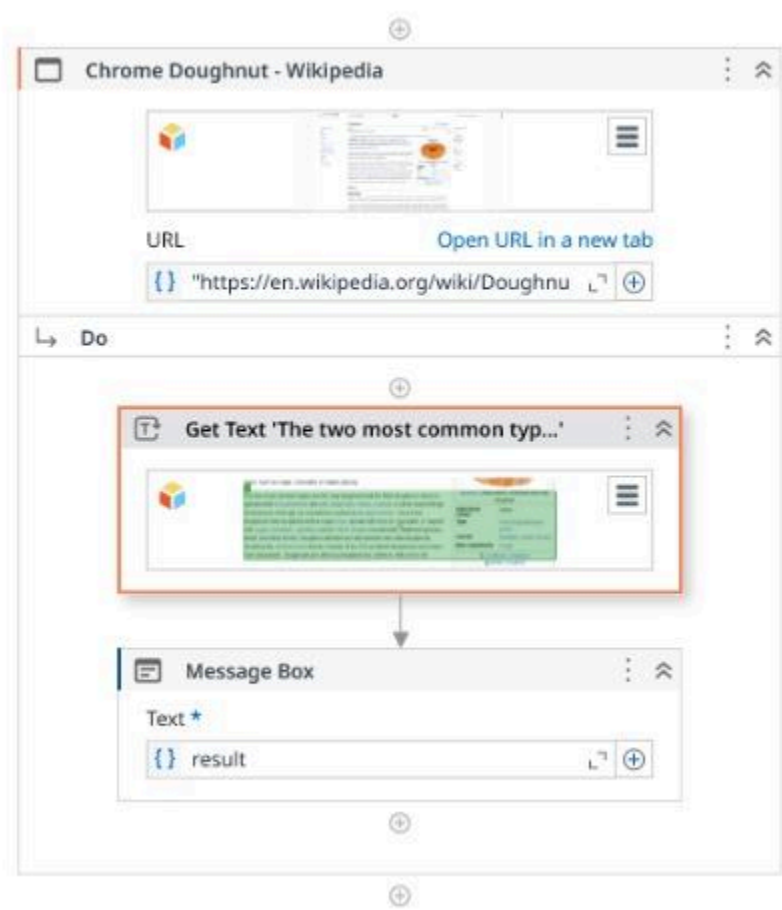
AIM:

To create a scream scraping using UiPath.

ALGORITHM:

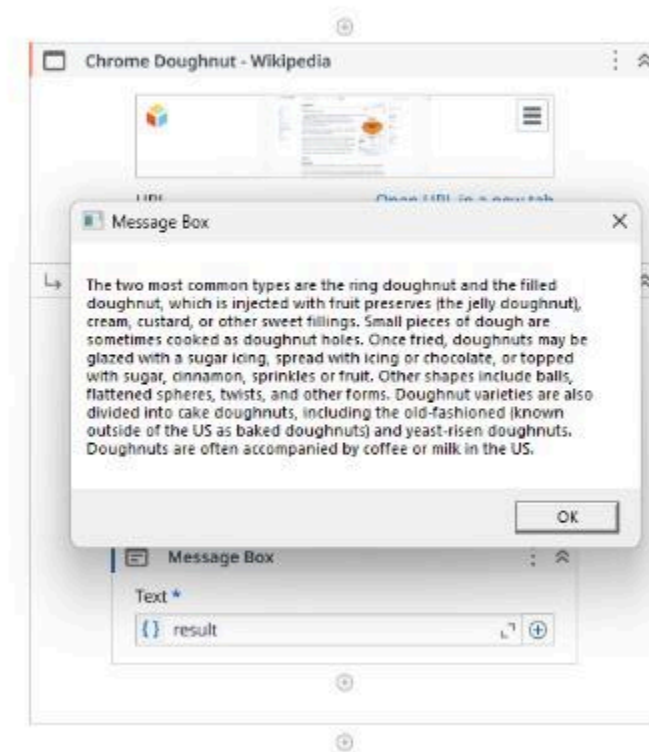
- Step 1:** Start the UiPath automation process and create a new workflow.
- Step 2:** Add the Use Application/Browser activity to open the required web page.
- Step 3:** Navigate to the web page containing the text to be extracted.
- Step 4:** Create a String variable named result to store the extracted text.
- Step 5:** Add the Get Text activity inside the Use Application/Browser activity.
- Step 6:** Indicate the required text or screen element using the Get Text activity.
- Step 7:** Set the Text Value output property of the Get Text activity to the result variable.
- Step 8:** Add a Message Box activity and display the extracted result variable.
- Step 9:** Run the workflow and verify the screen-scraped text output.

WORKFLOW:



Data Manager	Output	Markers	Terminal	Errors
{x} Variables	{*} Arguments	Namespaces	Connections	
Name	Data Type	Scope	Default Value	
Create variable				
{x} result	String	Use Application/Browser	{}	

OUTPUT:



RESULT:

Thus, the program has been executed successfully and output is verified.

4. Demonstrate Data manipulation using Contains method in UiPath.

AIM:

To demonstrate data manipulation using contains method in UiPath.

ALGORITHM:

Step 1: Start the UiPath automation process and create a new workflow.

Step 2: Create a String variable named sentence.

Step 3: Create a Boolean variable named result.

Step 4: Add an Assign activity to store a sentence in the sentence variable.

Step 5: Add another Assign activity to perform the Contains method on the sentence.

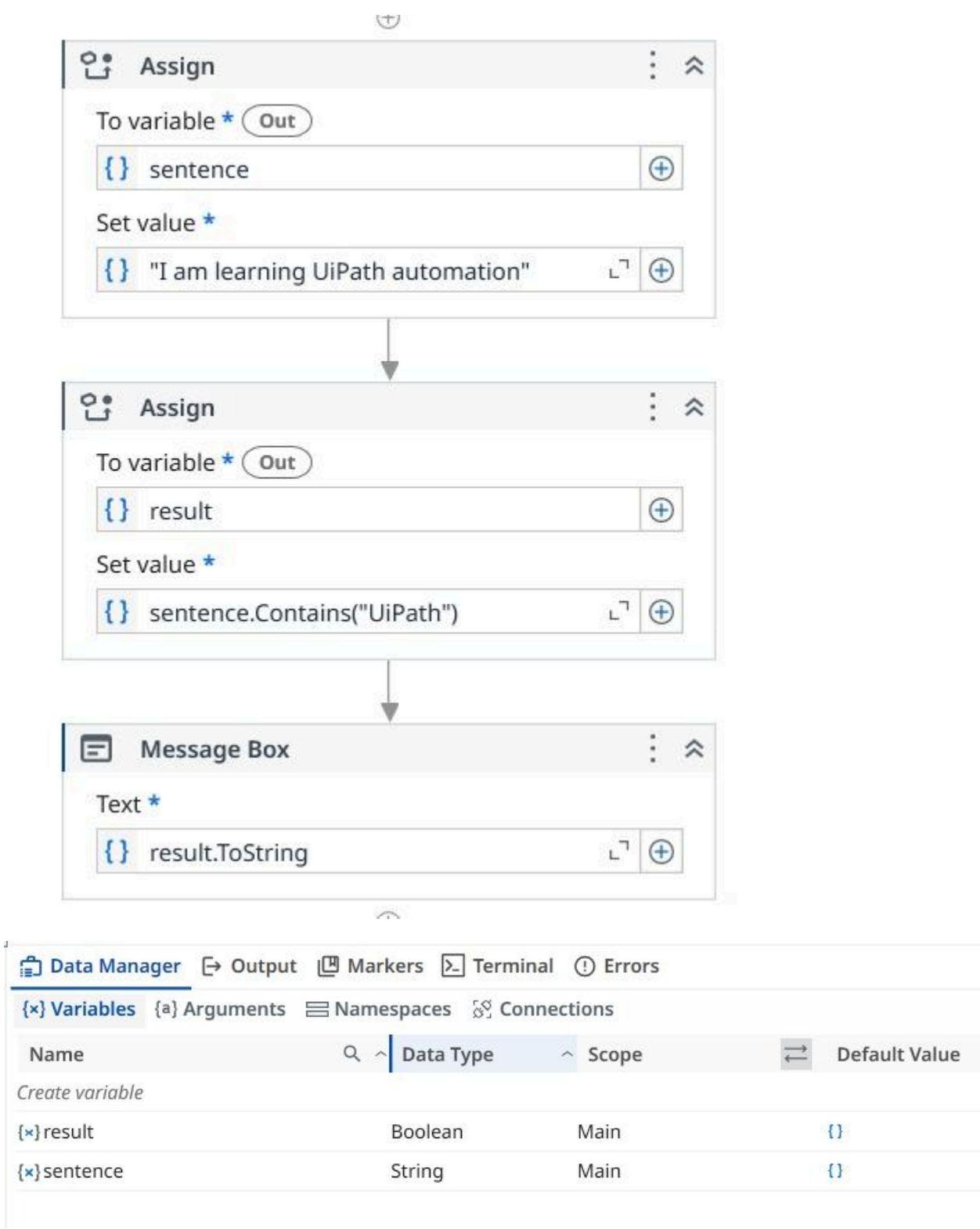
Step 6: Use the expression `sentence.Contains("UiPath")` to check whether the specified text is present.

Step 7: Store the Boolean result of the Contains method in the result variable.

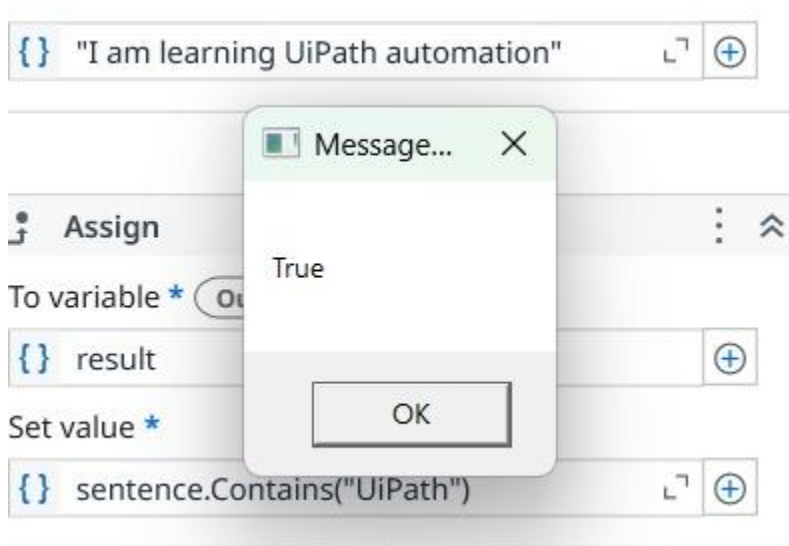
Step 8: Add a Message Box activity to display the value of result.

Step 9: Run the workflow and verify whether the specified text is present in the sentence.

WORKFLOW:



OUTPUT:



RESULT:

Thus, the program has been executed successfully and output is verified.

5. Implement sorting in the list with manipulating of data using Uipath.

AIM:

To implement sorting in the list with manipulation of data using UiPath.

ALGORITHM:

Step 1: Start the UiPath automation process and create a new workflow.

Step 2: Create a variable named MyList with the required String array data type.

Step 3: Initialize the MyList variable with the required list of items.

Step 4: Add an Assign activity to perform the sorting operation.

Step 5: Use the OrderBy method to arrange the list items in ascending order.

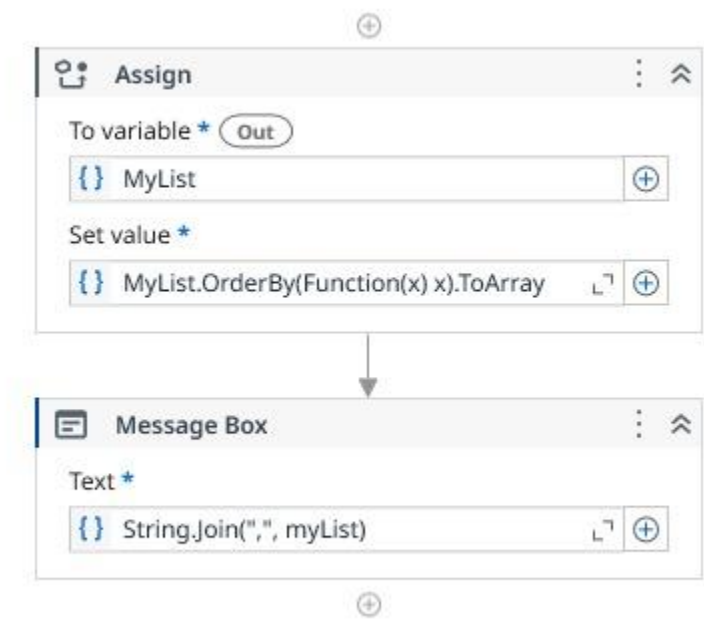
Step 6: Convert the sorted output into an array using the ToArray method.

Step 7: Store the sorted list back into the MyList variable.

Step 8: Add a Message Box activity to display the sorted list using the String.Join method.

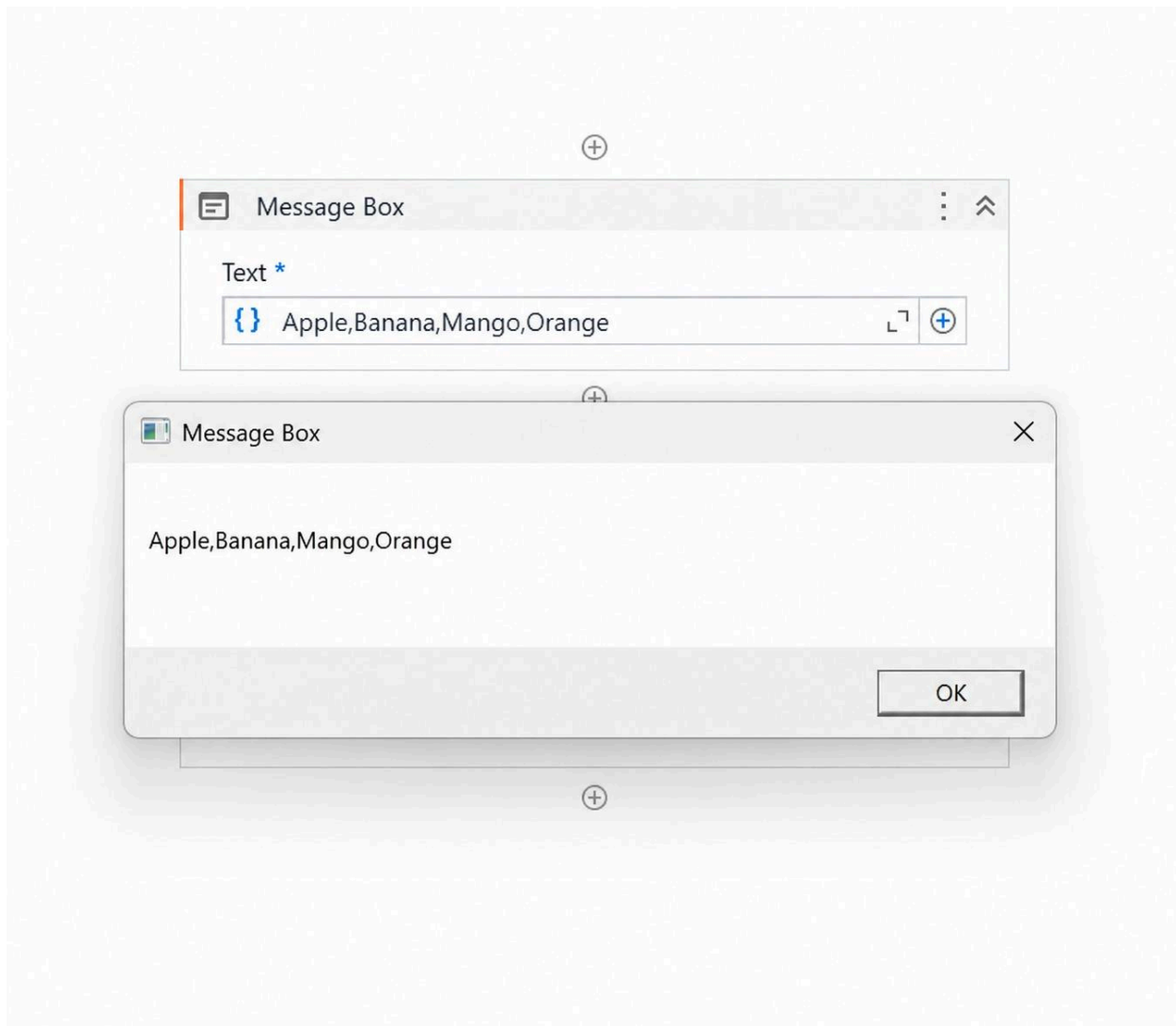
Step 9: Run the workflow and verify the sorted output.

WORKFLOW:



Data Manager			
Output			
Markers			
Terminal			
Errors			
Variables			
Arguments			
Namespaces			
Connections			
Name	Data Type	Scope	Default Value
Create variable			
(*) myList	String[]	Main	() ("Banana","Apple","Mango","Orange")

OUTPUT:



RESULT:

Thus, the program has been executed successfully and output is verified.

Ex No:06	EXCEPTION HANDLING
Date:	

AIM:

To write a program to demonstrate Exception Handling in UiPath.

ALGORITHM:

STEP 1: Start the process.

STEP 2: Open UiPath Studio and create a new process.

STEP 3: Drag and Drop try Catch activity to the workspace.

STEP 4: Drag and drop assign activity and create a variable result and in value give num \ den.

STEP 5: In catch browse for divide by zero exception.

STEP 6: And it catches then drag and drop message box and in text type “error”.

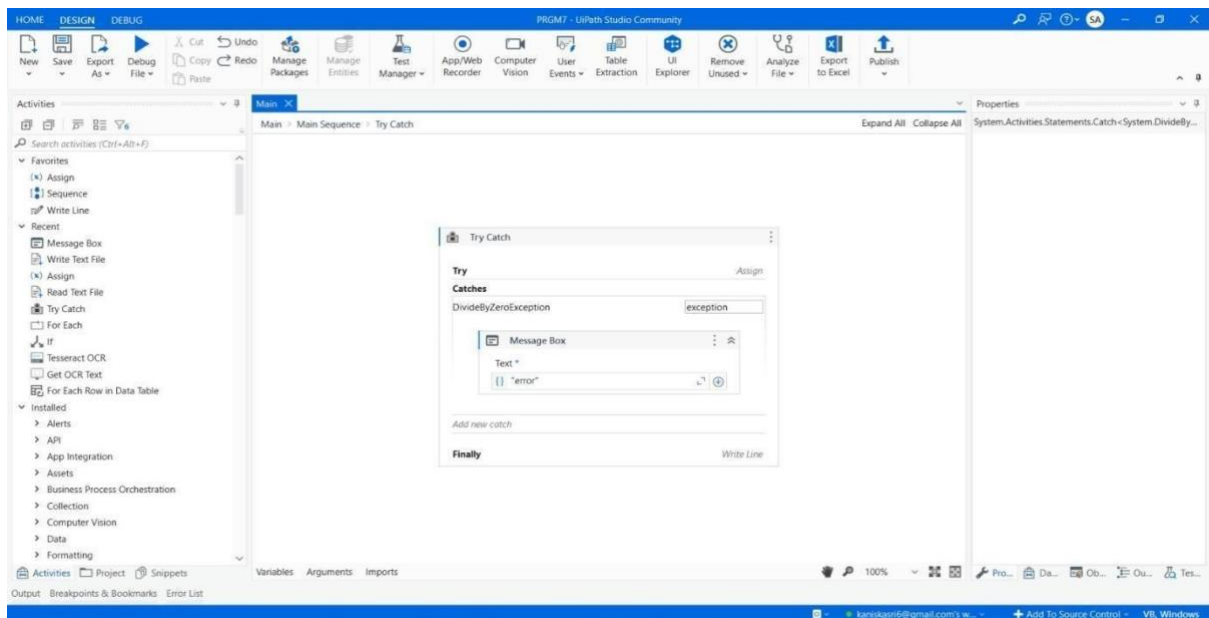
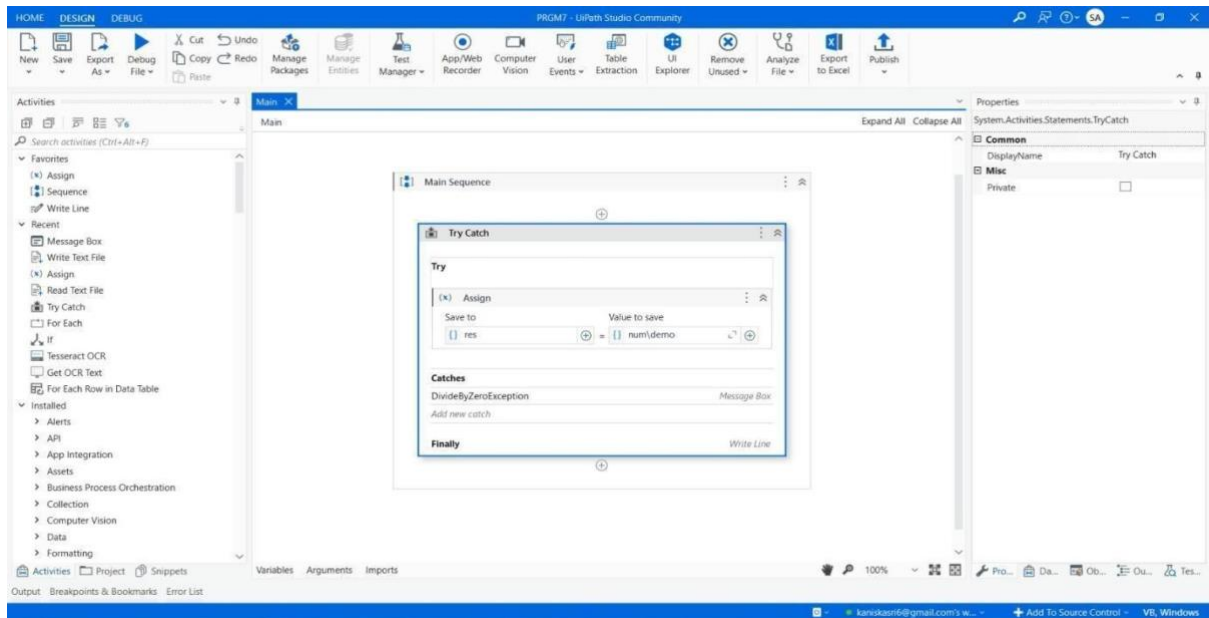
STEP 7: And in finally drag and drop writeline and in text type “completed”.

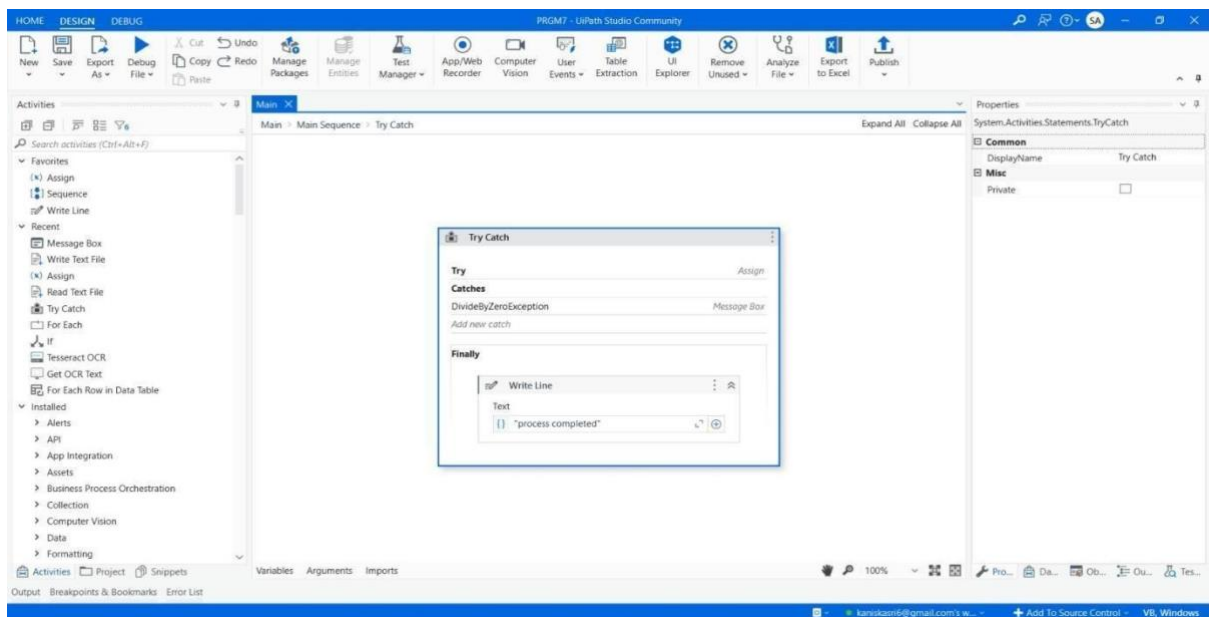
STEP 8: Then create a variable name called num and den in default give the number as 34, 2.

STEP 9: Save the program and run the program.

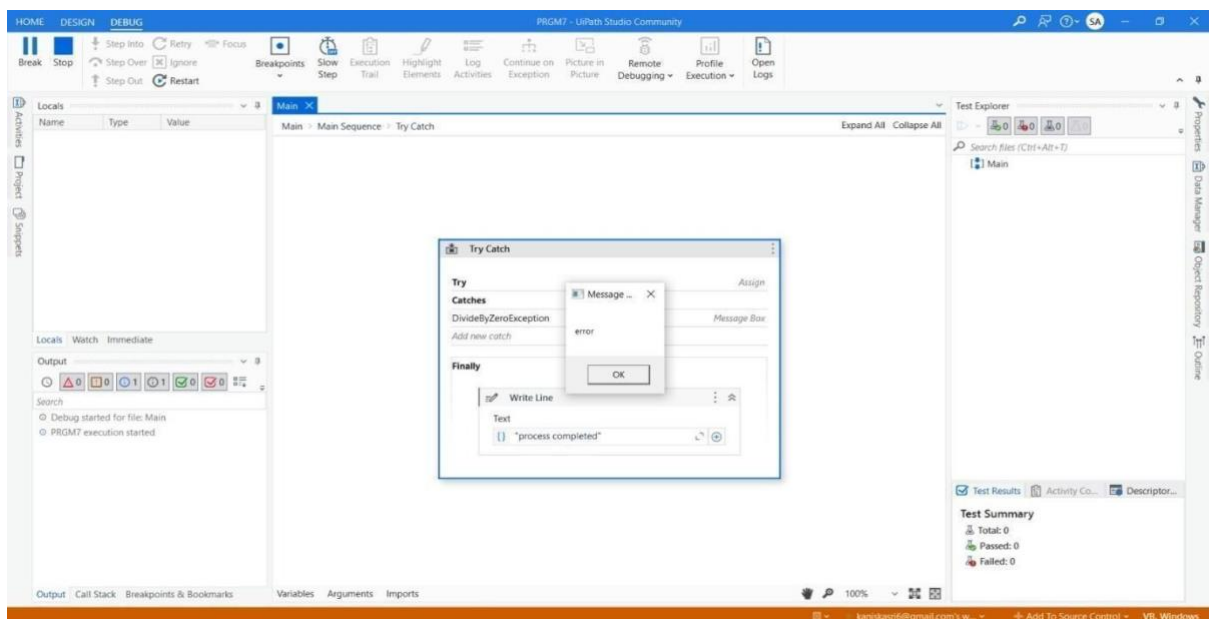
STEP 10: Stop the Process.

WORKFLOW:





OUTPUT:



RESULT:

Thus, the program is executed successfully and the output is verified.

Ex No:07	BOT FOR CLIPBOARD MANAGEMENT
Date:	

AIM:

To design a Bot for clipboard management.

.

ALGORITHM:

STEP 1: Start the UiPath workflow.

STEP 2: Create two string variables: userText and clipboardText.

STEP 3: Display an **Input Dialog** to ask the user to enter text.

STEP 4: Store the entered text in the userText variable.

STEP 5: Use the **Set Clipboard** activity.

STEP 6: Copy the value of userText to the Windows clipboard

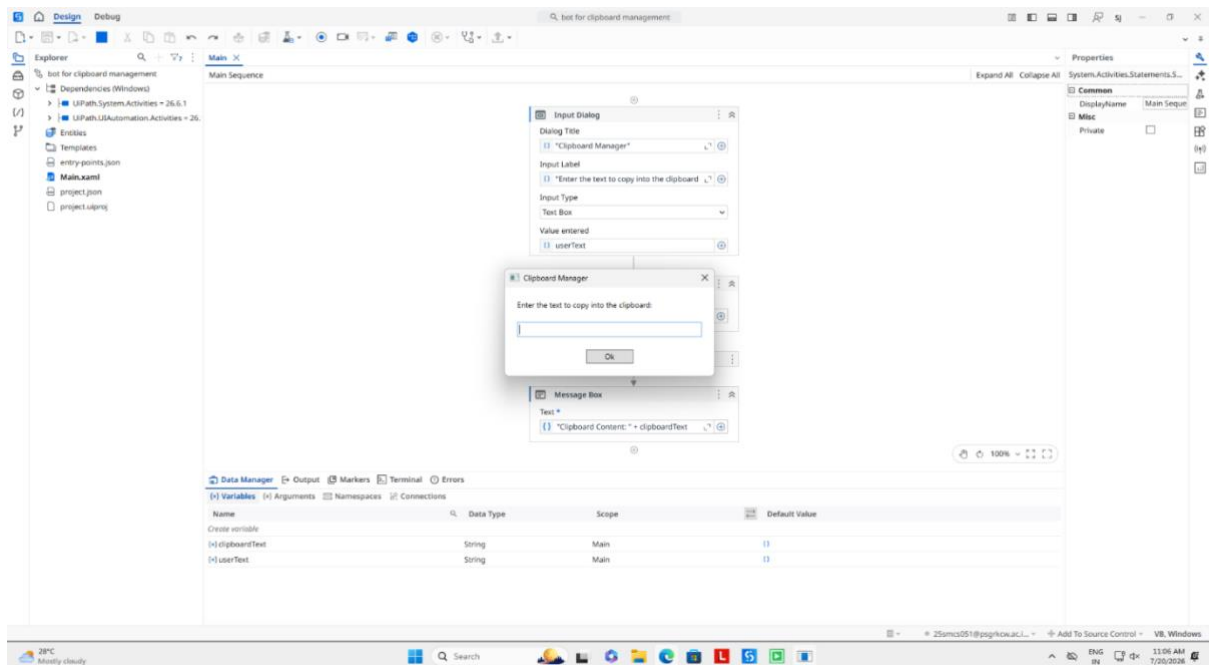
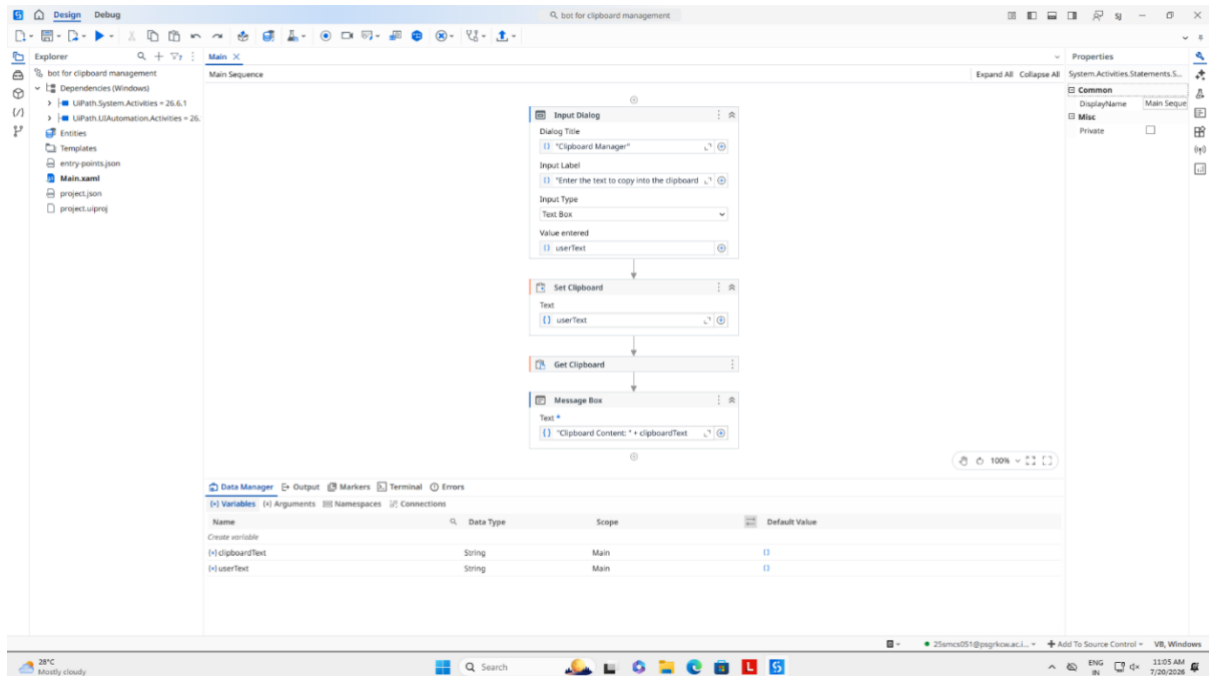
STEP 7: Use the **Get Clipboard** activity to retrieve the clipboard contents.

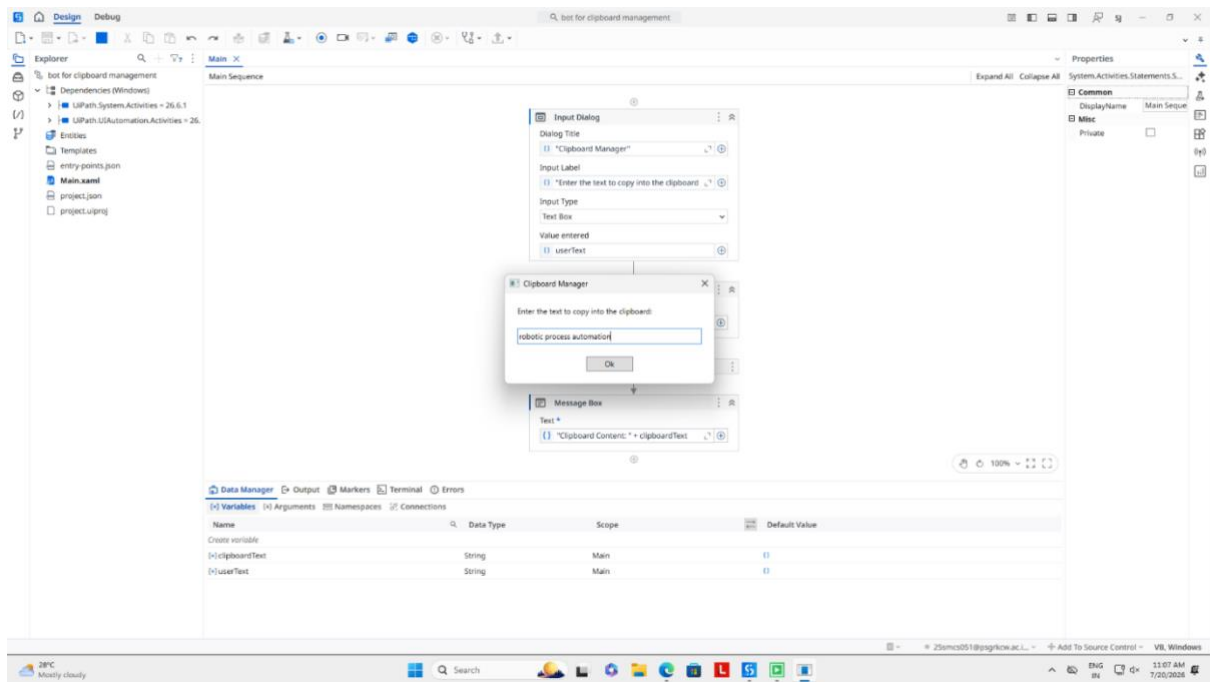
STEP 8: Store the retrieved clipboard content in the clipboardText variable.

STEP 9: Display the message "**Clipboard content:** " + clipboardText using a **Message Box**.

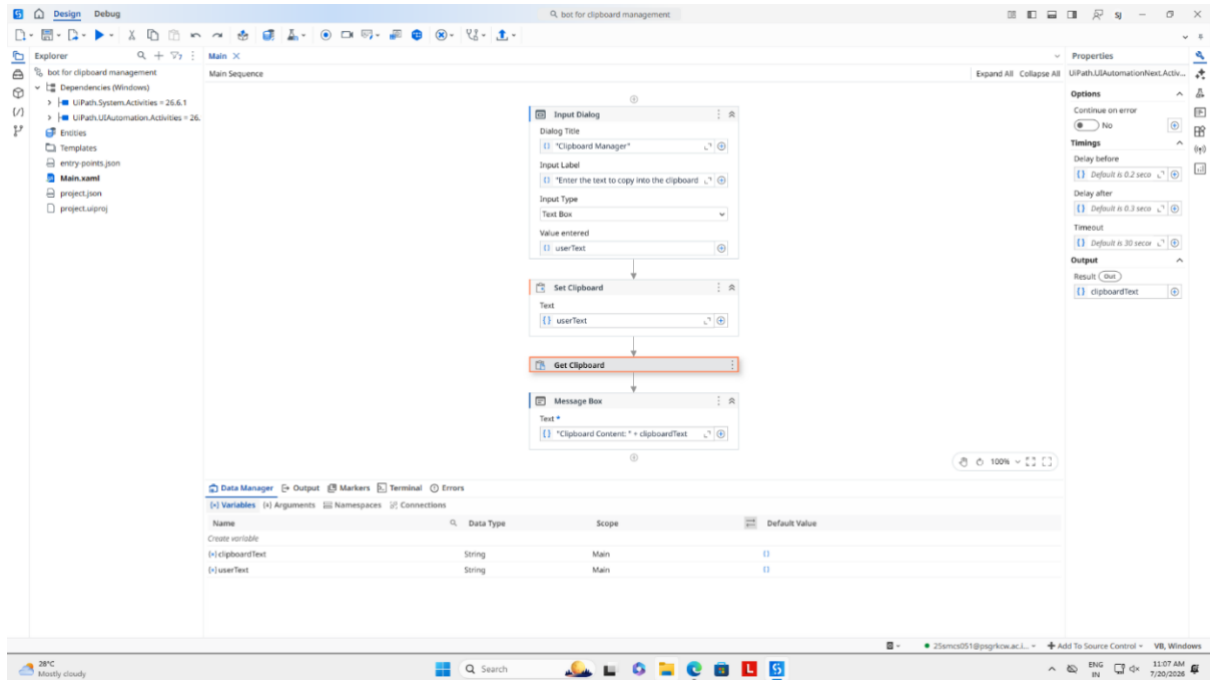
STEP 10: Stop the Process.

WORKFLOW

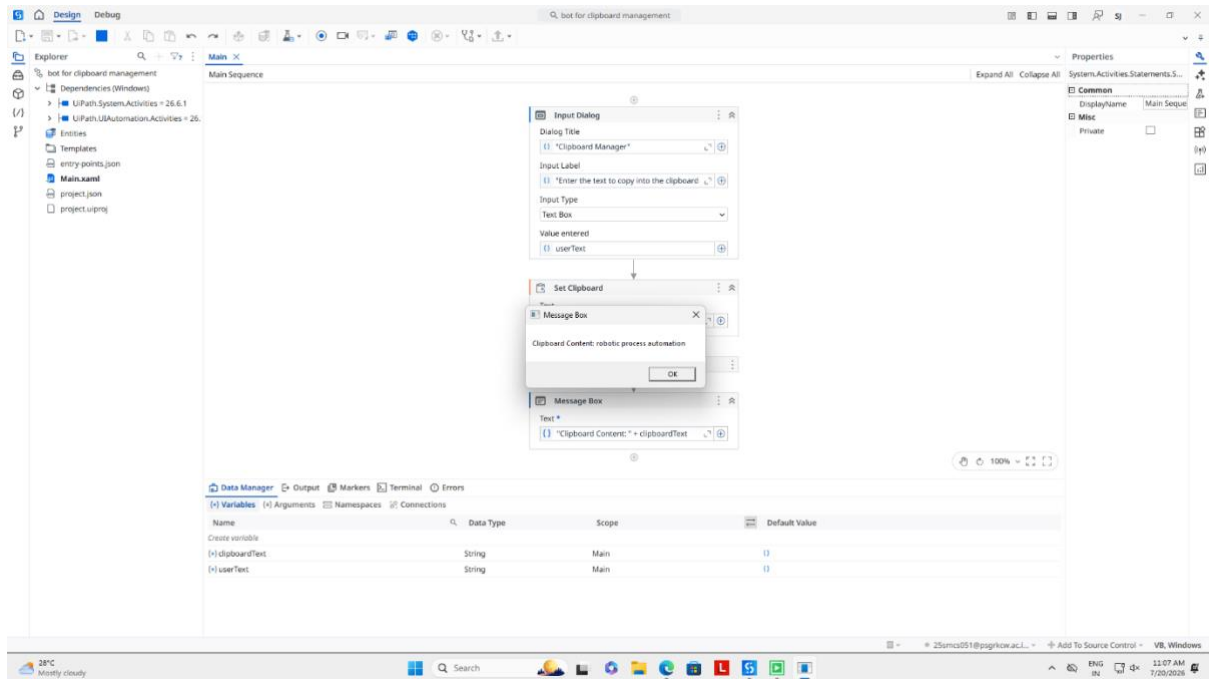




?



OUTPUT:



RESULT:

Thus, the program is executed successfully and the output is verified.

EX NO : 08 DATE :	DEMONSTRATE THE INVOKE METHOD ACTIVITY TO PERFORM OPERATIONS IN UIPATH
------------------------------------	---

AIM

To demonstrate the use of the Invoke Method activity in UiPath to call and execute a method for performing operations on data or objects.

ALGORITHM

STEP 1: Start the process.

STEP 2: Open UiPath and create a process.

STEP 3: In the main sequence add the invoke method.

STEP 4: Click the WriteLine for the math operation.

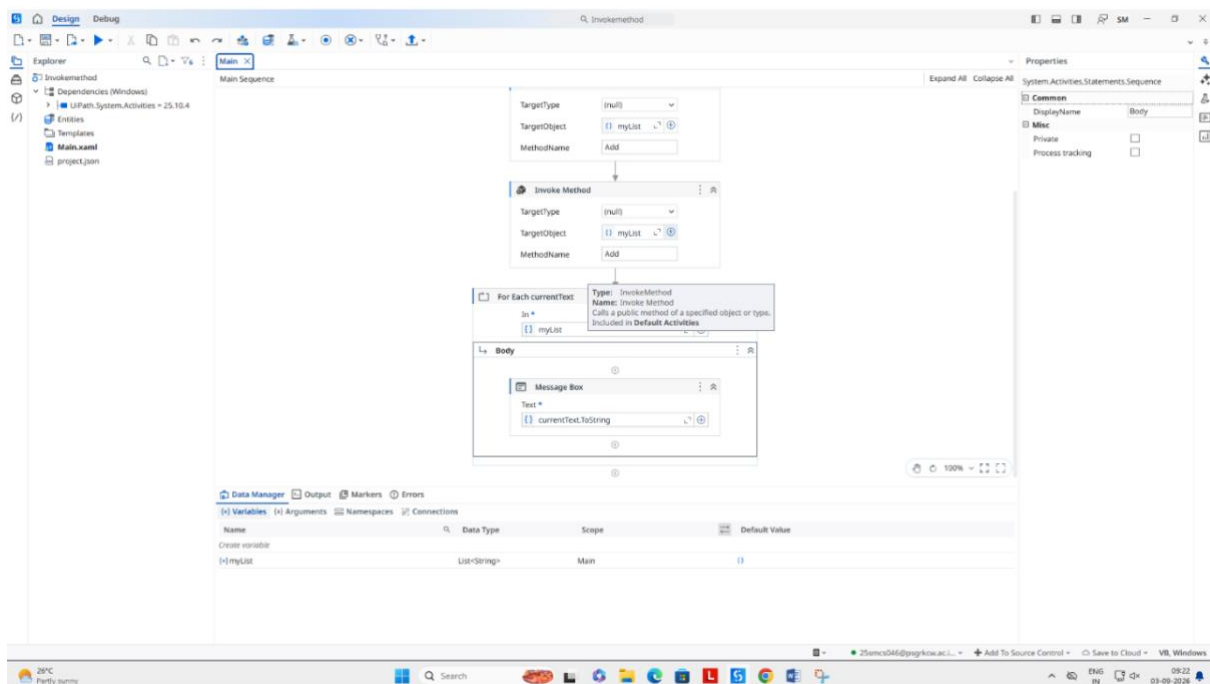
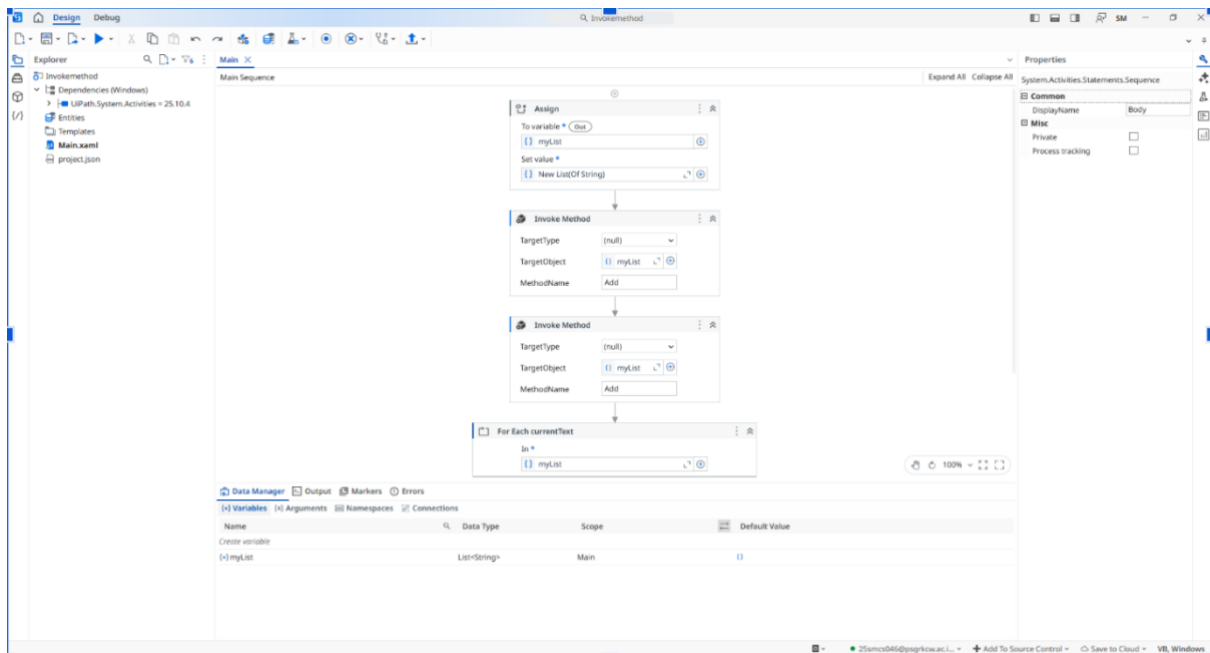
STEP 5: To perform string operation, use null value as target type.

STEP 6: Create variables to perform both string and math operations.

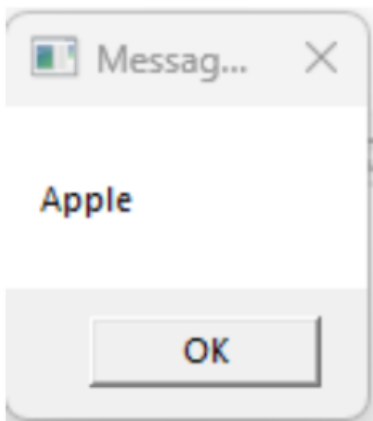
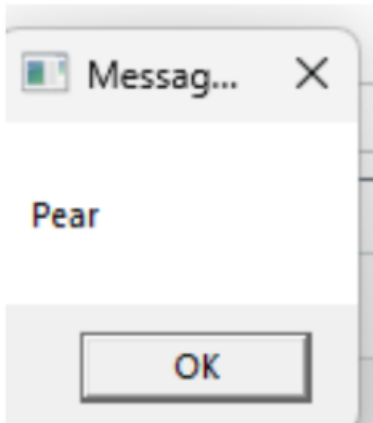
STEP 7: It uses the sting method to replace the function.

STEP 8: Save the process and run the program.

WORKFLOW



OUTPUT



RESULT

Thus the above was successfully executed and the output was verified.

EX NO : 09 DATE :	SECURE THE TEXT FROM THE USER
------------------------------------	--------------------------------------

AIM

To design a UiPath process that securely accepts a password from the user using the Input Dialog activity with the IsPassword option enabled, ensuring the entered text is hidden while typing.

ALGORITHM

STEP 1: Start the process.

STEP 2: Open UiPath Studio and create a new process.

STEP 3: In the Main Sequence, drag and drop the Input Dialog activity.

STEP 4: Set the Dialog Title as "Secure Input".

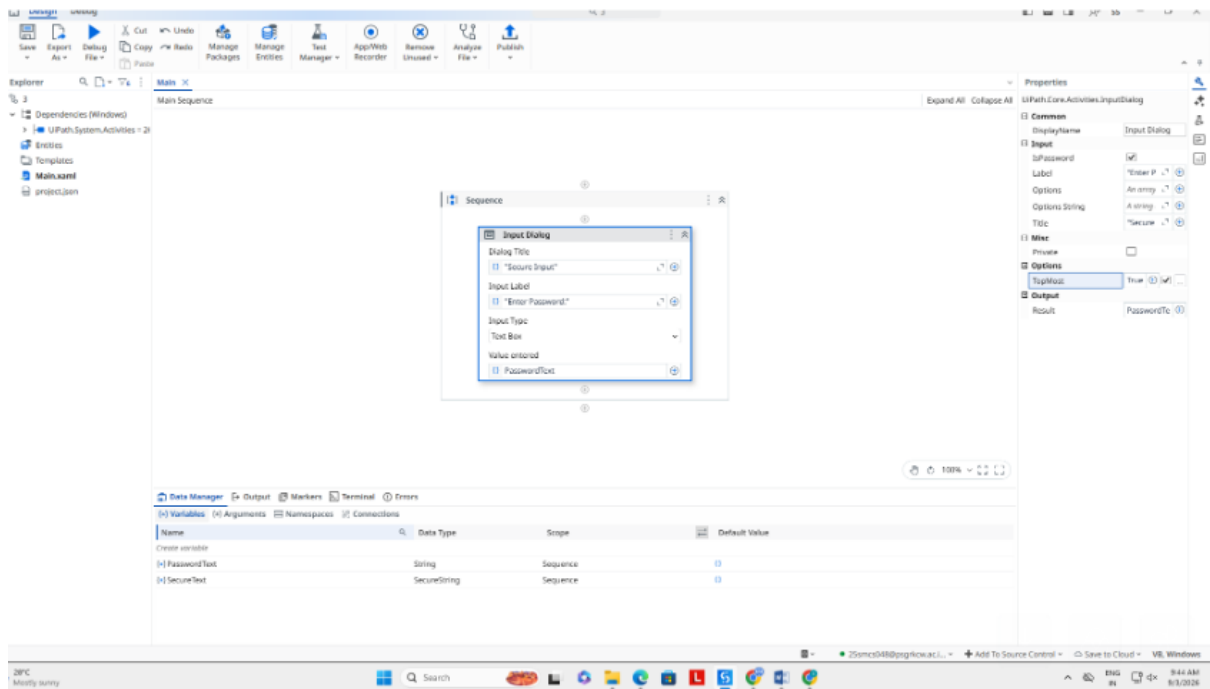
STEP 5: Set the Input Label as "Enter Password:".

STEP 6: Enable the IsPassword property to hide the text entered by the user.

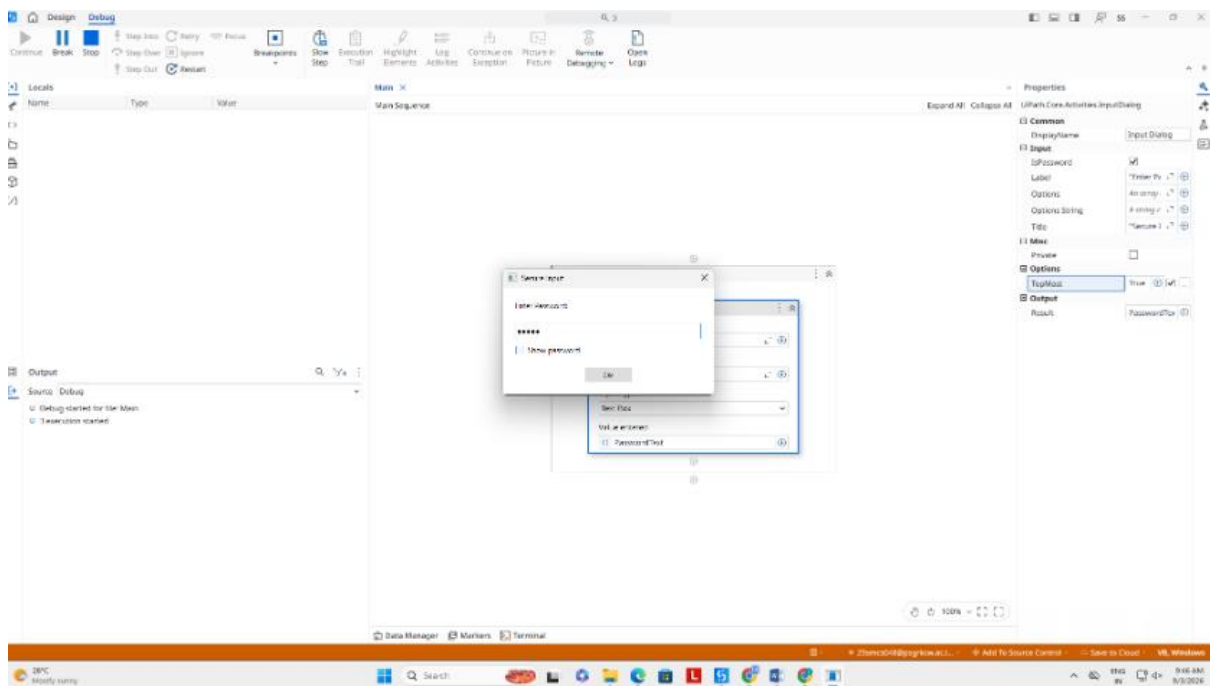
STEP 7: Create a String variable (e.g., PasswordText) and assign it to the Result property of the Input Dialog.

STEP 8: Save the process, run the workflow, enter the password, and click OK. The entered password is securely stored in the variable.

WORKFLOW



OUTPUT



RESULT

Thus the above was successfully executed and the output was verified.

